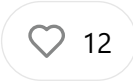


How to Implement Context Engineering Strategies for your Agent (Claude Code)

#59 | Managing context growth, preventing early stopping, and measuring what works



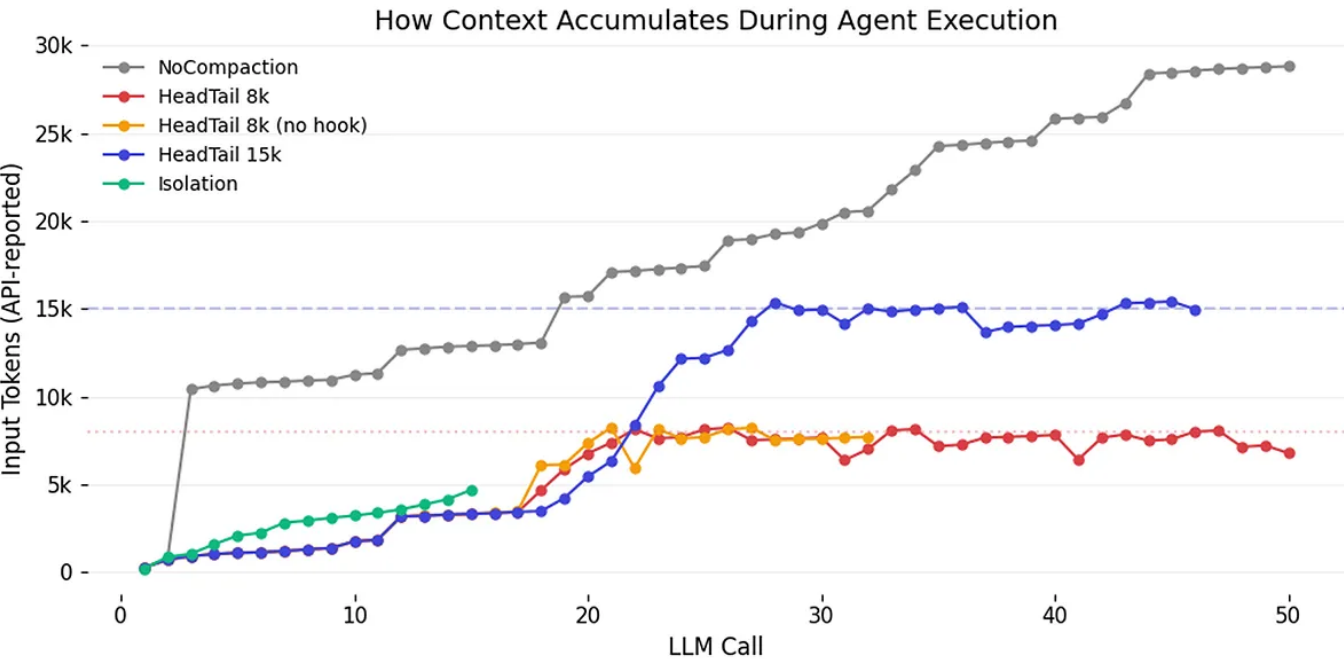
VICTOR DIBIA, PHD
MAR 11, 2026 · PAID



Context Engineering 101: How Agents Like Claude Code Manage Their Context Window



newsletter.victordibia.com



Context growth is different depending on the context engineering approach implemented on the same code review task (44-file repository, gpt-4.1-mini). Context grows monotonically without any strategy; HeadTail context compaction strategies show sawtooth patterns at their budget thresholds; Isolation shows bounded tokens per coordinator call.

get there - roughly \$1.80 on a frontier model. Run that 50 times a day across a team, and you're looking at \$2,700/month on a single agent workflow.

Most benchmarks for general-purpose agents like Claude Code and GitHub Copilot focus on task completion - did it get the job done? But in practice, *how* got the job done matters just as much. The agent that reads every file in the directory when it only needed three. The agent that compacts its context, drops critical information, then re-reads the same files. The agent that carries 50,000 tokens of stale tool results into every LLM call because nothing told it to forget. These agents might succeed, but they succeed expensively - and for indie developers watching their API bill, or businesses looking to [run agents at scale](#) context management can reduce token costs significantly, though often with quality tradeoffs that need to be understood.

Context in agents is cumulative - every message, tool result, and model response from previous steps gets carried forward into each new LLM call. As underlying models improve, agents can work on longer-horizon tasks ([METR benchmarks](#) show frontier models completing tasks equivalent to ~70 hours of human work 80% reliability, with task complexity doubling roughly every 7 months). But long tasks mean more accumulated context, and that creates three problems: the context window fills up and requests get rejected, token costs scale with context size, and model performance degrades as context grows ([Liu et al., 2023](#)).

Context engineering is the discipline of managing what goes into the context window. While we see the occasional "Claude is compacting" message, or the post on how note taking or even skills can help with context, the exact details on how to implement these strategies or how they impact performance are often vague. In this post, I break down the core strategies for context engineering, how to implement them, and their tradeoffs based on a benchmark I ran across five agent configurations.

TLDR; this post covers:

- The problem of context explosion in multi-step agent tasks
- Three core strategies for context engineering: compaction, isolation, and agentic memory
- A benchmark comparing these strategies on a code review task, showing the tradeoffs between token cost
- Key takeaways for when and how to apply context engineering in your agent

Note: For low complexity tasks, context engineering is less critical. Premature optimization can also hurt performance.

This post is adapted from my book [Designing Multi-Agent Systems](#), where I cover context engineering patterns in Chapter 4 alongside agent architecture, memory systems, and multi-agent coordination.



More Context ≠ Good Context

To understand why context engineering matters, the critical first step is to build up the right intuition for *how context grows during agent execution*. You can do this via *context inspection*.

Importantly, recall [the agentic loop](#) - agents address tasks by making LLM calls, interpreting the response, calling tools, and repeating. By default, all previous messages and tool results get appended to the context window for every new LLM call.

For example, here's what happens to a naive agent running a multi-step research task. The agent appends every message and tool result to its context:

```
1  Iteration 1:    888 tokens  (system + user message)
2  Iteration 2:  3,400 tokens  (+ list_directory result)
3  Iteration 3:  8,900 tokens  (+ read_file: context.py)
4  Iteration 4: 14,200 tokens  (+ read_file: compaction.py)
5  Iteration 5: 18,900 tokens  (+ grep + read_file: _agent.py)
```

Token growth from a PicoAgents benchmark run (debug_investigation task, gpt model, compaction). Each iteration adds tool results to the message history.

Interesting, the problem isn't just the hard limit on what fits in the context window (most LLMs now support very large windows). Rather, research shows LLMs struggle with large contexts even when they technically fit:

- [Lost in the middle](#): Information in the middle of long contexts gets less attention than information at the beginning or end ([Liu et al., 2023](#))
- **Performance degradation**: Accuracy can drop significantly based on how

information is positioned, not just whether it fits

- **Working memory:** The context window functions as the agent's working memory. But unlike RAM, adding more data can actively degrade retrieval of existing data. Context engineering curates what stays in that window

More context doesn't always mean better results.

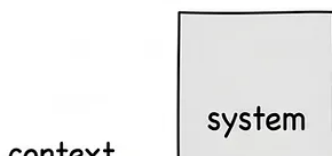
What Is Context Engineering?

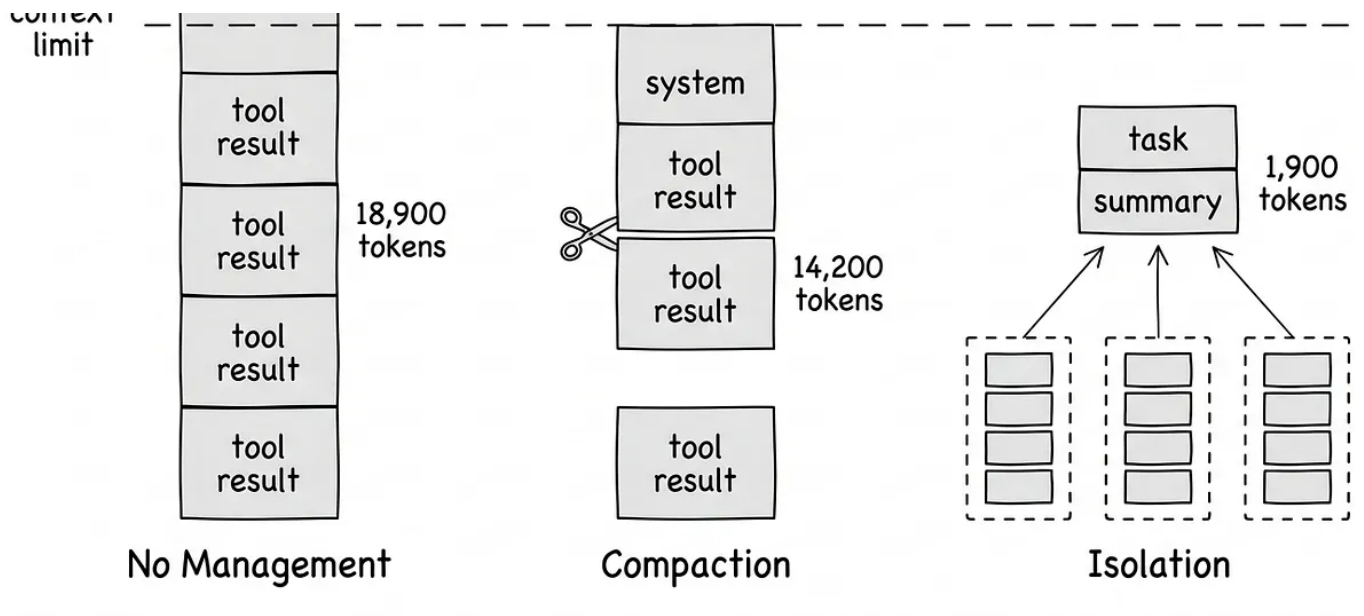
Context engineering is a set of techniques for managing what goes into the context window, *as the agent runs*. It answers four key questions:

1. **What to include** - Selecting relevant information
2. **What to exclude** - Trimming noise and redundancy
3. **How to represent it** - Compressing information efficiently
4. **Where to put it** - Positioning for model attention

[Andrej Karpathy](#) describes it as “the delicate art and science of filling the context window with just the right information for the next step.” Many agent failures trace back to context problems; the model had the right capabilities, but the wrong information was in (or missing from) the context window.

Three Core Strategies





We can categorize context engineering techniques into three core strategies:

Strategy 1: Compaction (Reactive Trimming)

Compaction reduces context by trimming, summarizing, or selectively retaining messages. It happens *just before* an LLM call in the agentic loop, triggered by a condition.

From the agent user's perspective, you configure a context strategy when creating the agent:

```

1  from picoagents import Agent
2  from picoagents.compaction import HeadTailCompaction, SlidingWindowCompaction
3
4  # Agent with head+tail compaction - keeps task definition + recent work
5  agent = Agent(
6      name="investigator",
7      instructions="Investigate the codebase and find the bug.",
8      model_client=model_client,
9      tools=[read_file, list_directory, grep],
10     compaction=HeadTailCompaction(
11         token_budget=100_000, # Compact when context exceeds this
12         head_ratio=0.2,      # 20% for task context, 80% for recent work
13     ),
14     max_iterations=20,
15 )
16
17 response = await agent.run("Find why the auth middleware fails on refresh tokens.")

```

Under the hood, the agent loop applies this strategy before each LLM call:

```

1  while not done:
2      # Check if compaction needed before calling the model
3      if context_exceeds_budget(messages): # e.g., > 80% of limit
4          messages = compact(messages)
5
6      response = call_llm(messages)
7      # ... handle response, execute tools, append results

```

If you've used coding agents (Claude Code, GitHub Copilot CLI, etc.), you may occasionally see the agent pause, mention “compacting”, summarize prior steps and continue. This is the compaction strategy. The agent hit a threshold (e.g., 80% the context window, or a hard cap like 100k tokens) and triggered compaction before the next LLM call. The budget determines *when* to compact; the approaches below determine *how*:

Sliding Window: Keep system message + most recent messages that fit in budget.
Simplest approach.

```

1  # From picoagents/compaction.py - SlidingWindowCompaction.compact()
2  def compact(self, messages):
3      current_tokens = self._count_tokens(messages)
4      if current_tokens <= self.token_budget:
5          return messages # No compaction needed
6
7      groups = self._find_atomic_groups(messages) # Keep tool calls with results
8
9      # Always keep system message
10     system_groups, system_tokens = [], 0
11     if groups and messages[groups[0][0]].role == "system":
12         system_groups.append(groups[0])
13         system_tokens = self._count_tokens([messages[i] for i in groups[0]])
14         groups = groups[1:]
15
16     # Fill from end with remaining budget
17     remaining_budget = self.token_budget - system_tokens
18     kept_groups, kept_tokens = [], 0
19
20     for group in reversed(groups):
21         group_tokens = self._count_tokens([messages[i] for i in group])
22         if kept_tokens + group_tokens <= remaining_budget:
23             kept_groups.insert(0, group)
24             kept_tokens += group_tokens
25         else:
26             break # No more room
27
28     kept_indices = set()
29     for group in system_groups + kept_groups:
30         kept_indices.update(group)
31     return [messages[i] for i in sorted(kept_indices)]
32

```

Head+Tail: Split the budget between head (system prompt, initial task) and tail (recent work). Drop middle messages. This preserves both the task definition and recent progress.


```

1  # From picoagents/compaction.py - HeadTailCompaction.compact()
2  # Split budget: 20% head (task context), 80% tail (recent work)
3  head_budget = int(self.token_budget * self.head_ratio)
4  tail_budget = self.token_budget - head_budget
5
6  # Fill head from start, tail from end, drop middle
7  for group in groups:
8      if head_tokens + group_tokens <= head_budget:
9          head_groups.append(group)
10     else:
11         break
12
13  for group in reversed(remaining_groups):
14      if tail_tokens + group_tokens <= tail_budget:
15          tail_groups.insert(0, group)
16     else:
17         break
18

```

Tool Result Clearing: Once a tool has been called deep in history, clear the raw result but keep the message structure. Anthropic recently launched this on the [Claude Developer Platform](#) - it's the lightest-touch form of compaction.

Summarization: Instead of dropping old messages, compress them into a summary using a *fast* model. Preserves information at the cost of an extra LLM call. It can also lack resolution - the summary may omit details needed later. In benchmark described below, there is the concept of “thrashing” - the agent repeats steps because the compaction strategy dropped information it needed, causing it to re-read files or re-run tools.

Semantic Selection: Use embeddings to select contextually relevant messages rather than just the most recent. More expensive but can retain relevant older information that sliding window would drop. [LangGraph](#) and [Mem0](#) provide implementations.

Limitation: All compaction is reactive - context grows, then you trim. Even semantic selection responds to accumulated context. If the agent needs something that was trimmed or poorly summarized, that information is gone.

Strategy 2: Isolation (Architectural Prevention)

Run sub-tasks in separate contexts. Only the summary of steps enters the main agent's context.

```
1  # Define a sub-agent with its own tools and compaction
2  sub_agent = Agent(
3      name="code_reviewer",
4      description="Reviews code in a directory",
5      instructions="Read all .py files and document classes and functions.",
6      model_client=model_client,
7      tools=[read_file, list_directory],
8      compaction=HeadTailCompaction(token_budget=50_000, head_ratio=0.2),
9      max_iterations=20,
10 )
11
12 # Wrap it as a tool – coordinator delegates, sees only the summary
13 coordinator = Agent(
14     name="coordinator",
15     instructions="Delegate each directory to the code_reviewer tool.",
16     model_client=model_client,
17     tools=[sub_agent.as_tool()], # Agent becomes a callable tool
18     max_iterations=15,
19 )
20
21 response = await coordinator.run("Review the repository.")
22 # Coordinator context stays small; sub-agent context is discarded
```

How it works: A coordinator agent delegates tasks to sub-agents, each running its own context window. The sub-agents do the heavy lifting — reading files, calling tools, accumulating context — and only the final result crosses back. The intermediate work is discarded, so the coordinator's context stays bounded regardless of how much work the sub-agents do. In PicoAgents, `as_tool()` wraps

any agent as a callable tool to enable this pattern.

When to use:

- Tasks involve distinct sub-problems
- Sub-tasks generate lots of intermediate context
- You can define clear interfaces (input -> output)
- You want to parallelize work (multiple specialists can run concurrently)

This same principle applies temporally - some systems restart the agent in fresh sessions, passing only a summary forward instead of the full context.

[RelentlessAgent](#) takes this approach, running up to 10,000 sequential sub-sessions where each one receives only the original task plus a summary of prior work.

Strategy 3: Agentic Memory (External Storage)

Give the agent tools to explicitly manage its own memory *outside* the context window. The agent decides what to save, when to retrieve, and what to forget.

[Anthropic](#) calls this “structured note-taking” - the agent writes notes persisted external storage, then retrieves them when needed. Think of it like maintaining `NOTES.md` file. This enables tracking progress across complex work without keeping everything in active context.

```

1  # From picoagents/tools/_context_tools.py - TodoWriteTool
2  # Agent persists progress to disk, outside the context window
3
4  def _save_todos(todos):
5      path = _get_todo_path() # .picoagents/todos/session_*.json
6      path.parent.mkdir(parents=True, exist_ok=True)
7      data = {
8          "session_id": _get_session_id(),
9          "updated_at": datetime.now().isoformat(),
10         "todos": todos,
11     }
12     path.write_text(json.dumps(data, indent=2))
13
14     # Agent calls todo_write to track progress externally
15     # This keeps task state out of the context window entirely
16

```

Key difference from compaction: Compaction trims what's already in context. Agentic memory moves information *outside* the context entirely - the agent retrieves it on demand.

[Agent Skills](#) work the same way but for procedural knowledge instead of facts. skill description costs ~100 tokens at startup; the full instructions (~2,000 token only enter context when the task matches. Agentic memory stores what the agent learned (facts, findings, progress); skills store how the agent should act (procedures, workflows). Both keep information outside the context window until it's needed. I walk through [implementing skills from scratch](#) in a separate post.

When to use:

- Long-running tasks where information needs to persist
- Tasks where the agent needs to selectively recall specific facts
- You want the agent to decide what's worth remembering

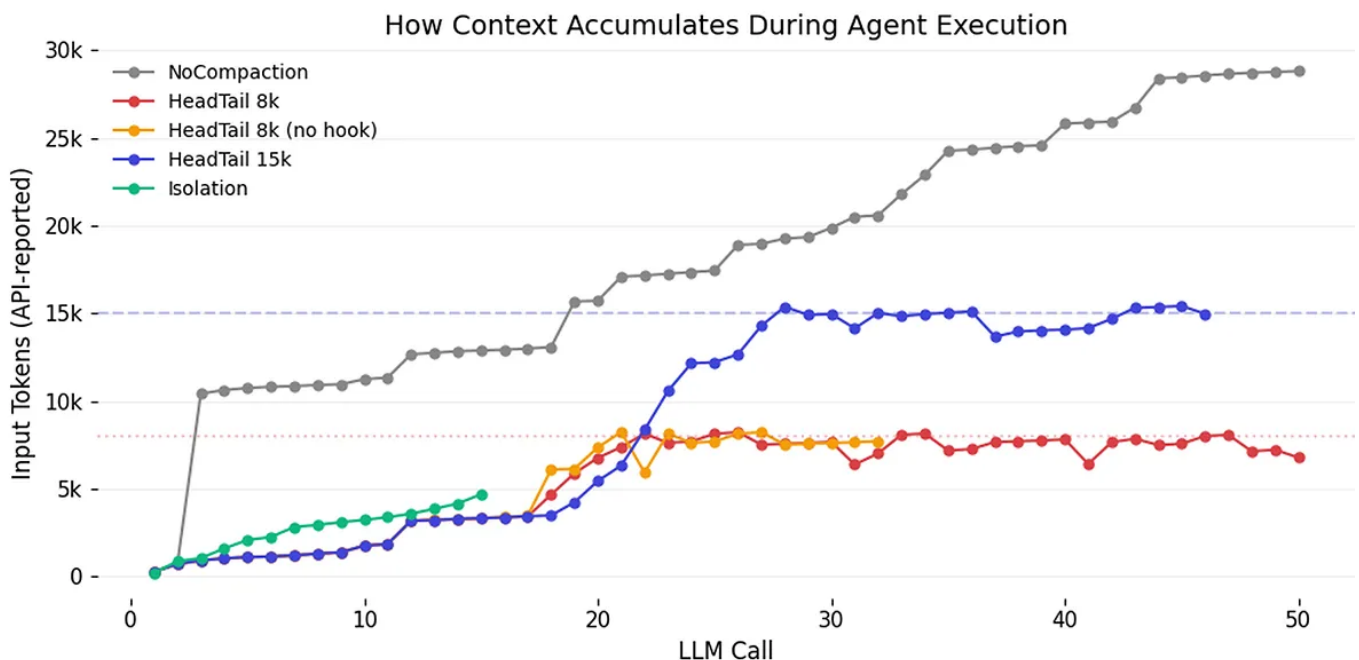
Benchmark Comparison

To illustrate the tradeoffs, I ran a code review task across five agent configurations using PicoAgents' evaluation system. The task: review a 44-file Python repository, read every .py file, document all classes and functions, and produce a quality assessment. All agents use the same model (gpt-4.1-mini), the same two tools (read_file, list_directory), and the same system prompt. The only variables are context strategy and whether a completion hook is enabled.

Context Engineering 101: How Agents Like Claude Code Manage Their Context Window



newsletter.victordibia.com



The five configurations:

- **No Compaction:** Full context carried forward, no trimming. Completion hook forces the agent to keep working until the task is done (up to 50 iterations).

- **HeadTail 8k:** Head+tail compaction with an 8k token budget (20% head, 80% tail). Same completion hook.
- **HeadTail 8k (no hook):** Same 8k compaction, but no completion hook; the agent stops when it does not call any additional tools.
- **HeadTail 15k:** Head+tail with a larger 15k token budget. Completion hook enabled.
- **Isolation:** A coordinator delegates to sub-agents via `as_tool()`. Each sub-agent has its own 50k HeadTail budget. No completion hook on the coordinator.

Strategy	Input Tokens	LLM Calls	Files Read	Duplication	Quality (0-10)
No Compaction	915,591	50	16	27%	6.0
HeadTail 8k	276,390	50	14	55%	4.0
HeadTail 8k (no hook)	144,136	32	17	15%	5.5
HeadTail 15k	402,102	46	14	46%	5.1
Isolation	39,759 + 79,065	15 + 51	—	—	4.5

Results from a single run per config using PicoAgents evaluation (handtracking repository, 44 .py files). Quality scored by an LLM judge (gpt-5.2-chat) on completeness and actionability. All hooked agents use LLMCompletionCheckHook with max_restarts=5

The completion hook fires when the agent would normally stop (no more tool calls). It summarizes the conversation so far, asks a judge LLM “is this task complete?”, and if not, injects a message telling the agent to continue. This gives the agent *cognitive durability* — the ability to persist through long tasks but adds iterations (and tokens) each time it restarts.

A note on evaluation: Quality scores come from an LLM-as-judge, which introduces its own variance. Building confidence in these numbers requires

inspecting the judge's rationale for each score, ensuring the judge sees the full agent trace, and providing ground truth references (e.g., the repository has 44 .ipynb files; did the agent find them?). With single runs, treat the scores as directional signals, not precise measurements. I'll cover LLM judge design in detail in a future post.

Several patterns emerge:

No compaction works, but it's expensive. NoCompaction scored highest (6.0) by brute-forcing - carrying full history across 50 iterations, 915k tokens. The agent never forgets what it read, so it avoids re-reads (only 27% duplication). The tradeoff is cost: 2-6x more tokens than compacted agents, and 22 minutes of wait time.



Continue reading this post for free, courtesy of Victor Dibia, PhD.

[Claim my free post](#)

Or purchase a paid subscription.